

Cross-Site Scripting (XSS)

Q1 - Propósito dos Tokens `__elgg_ts` e `__elgg_token`

As linhas que obtêm os tokens de segurança são:

```
var ts+"&__elgg_ts="+elgg.security.token.__elgg_ts;  
var token+"&__elgg_token="+elgg.security.token.__elgg_token;
```

Estes tokens são mecanismos de proteção contra ataques CSRF (Cross-Site Request Forgery) implementados pelo Elgg. A sua função é a seguinte:

`__elgg_ts` (Timestamp)

Representa o timestamp do momento em que a página foi carregada. O servidor verifica se o timestamp é recente para evitar ataques de replay, onde um atacante utiliza um pedido antigo interceptado.

`_elgg_token` (Token de Segurança)

É um token secreto único gerado pelo servidor para aquela sessão específica. O servidor associa este token à sessão do utilizador e rejeita qualquer pedido que não inclua o token correto. Isto impede que um site externo forme pedidos em nome do utilizador.

Porque são necessários no ataque XSS?

Sem estes tokens, o servidor Elgg rejeitaria o pedido de adicionar amigo com um erro de autenticação. O ataque XSS consegue contornar esta proteção CSRF porque o script malicioso é executado no contexto da página do próprio servidor (mesmo domínio), tendo assim acesso direto aos tokens através de:

```
elgg.security.token.__elgg_ts  
elgg.security.token.__elgg_token
```

Isto demonstra como o XSS é particularmente perigoso: permite contornar proteção CSRF porque o código malicioso é executado como se fosse código legítimo do próprio site, tendo acesso a todos os recursos da página incluindo tokens de segurança.

Q2 - Ataque com Apenas Modo Editor

Se o Elgg apenas disponibilizasse o modo Editor (sem modo Text/HTML), o ataque ainda seria possível, embora de forma diferente.

Problema com o modo Editor

O modo Editor (WYSIWYG) processa o texto introduzido e tipicamente remove tags `<script>` por razões de segurança de formatação. No entanto, não remove todos os atributos HTML que permitem executar JavaScript.

Solução alternativa - Event Handlers

É possível injetar JavaScript através de atributos de eventos HTML como onerror, onload ou onclick, que os editores WYSIWYG normalmente não filtram:

```

```

O browser tenta carregar a imagem com src="x", falha (porque não existe), e executa o código no atributo erro. O modo Editor normalmente permite a inserção de imagens e não filtra os eventos handlers, tornando o ataque igualmente possível.

Conclusão: O ataque XSS ainda seria bem sucedido mesmo sem o modo Texto, usando event handlers HTML em vez de tags script diretas. Isto demonstra que a defesa contra XSS não pode depender apenas de filtrar tags script.

Q3 - Modalidade do Ataque XSS

O ataque realizado (Samy Worm) enquadra-se na modalidade Stored XSS (XSS Persistente), pelas seguintes razões:

Características do Stored XSS

- O script malicioso foi guardado na base de dados do Elgg, no campo 'Brief Description' do perfil do Samy
- O script é servido automaticamente pelo servidor a qualquer utilizador que visita o perfil
- Não requer qualquer interação da vítima além de visitar a página - o script executa automaticamente com window.onload
- Afeta múltiplas vítimas - todos os utilizadores que visitam o perfil são afetados
- O script persiste no servidor até ser removido manualmente

Porque não é Reflected XSS

No Reflected XSS, o script viaja no URL do pedido e é refletido pelo servidor na resposta. Neste ataque, o script não está no URL - está guardado na base de dados e servido como conteúdo normal da página.

Porque não é DOM XSS

No DOM XSS, o script malicioso é injetado e executado diretamente no DOM do browser sem passar pelo servidor. Neste ataque, o script é armazenado e servido pelo servidor, sendo executado quando o HTML é renderizado.

Característica	Stored XSS	Reflected XSS	DOM XSS
Script guardado no servidor	Sim	Não	Não
Script no URL	Não	Sim	Não
Múltiplas vítimas	Sim	Apenas quem clica no link	Apenas quem clica no link
Este ataque	Sim	Não	Não

Q4 - Diferenças entre os 3 Websites CSP

Os três websites apresentam comportamentos distintos devido às suas diferentes configurações de CSP.

Área	Descrição	example 32a	example 32b	example 32c
1	Inline nonce 111	OK	Failed	OK
2	Inline nonce 222	OK	Failed	OK
3	Inline sem nonce	OK	Failed	Failed
4	From self	OK	OK	OK
5	From example 60	OK	OK*	OK
6	From example 70	OK	OK	OK

* Após modificação da configuração para incluir *.example60.com

example 32a - Sem CSP

Não tem qualquer política de segurança definida. O browser executa todos os scripts sem restrições, independentemente da sua origem ou de terem nonce. E completamente vulnerável a ataques XSS - qualquer script injetado seria executado.

example32b - CSP via Apache

A política CSP é definida no ficheiro de configuração do Apache apache.apache.csp.conf:

```
Header set Content-Security-Policy "  
    default-src 'self';  
    script-src 'self' *.example60.com *.example70.com  
"
```

O Apache adiciona automaticamente este header a todas as respostas do VirtualHost. Scripts inline (áreas 1, 2, 3) são bloqueados porque a política não inclui 'unsafe-inline' nem nonces. Apenas scripts do próprio servidor e dos domínios autorizados são permitidos.

example32c - CSP via PHP

A política CSP é definida programaticamente no ficheiro php index.php:

```
$cspheader = "Content-Security-Policy:".  
    "default-src 'self';".  
    "script-src 'self' 'nonce-111-111-111' 'nonce-222-222-222' "  
    "*.example60.com *.example70.com";
```

A principal diferença face ao exemplo 32b é a possibilidade de usar nonces. Scripts inline com os nonces corretos (111-111-111 e 222-222-222) são permitidos. Scripts inline sem nome (área 3) continuam bloqueados. Esta abordagem é mais flexível pois permite gerar nonces dinamicamente no código da aplicação.

Como o CSP previne XSS

O CSP previne ataques XSS de forma eficaz porque:

- Mesmo que um atacante consiga injetar um script malicioso na página, o browser recusa-se a executá-lo se não vier de uma fonte autorizada pela política
- Scripts inline são bloqueados por omissão, o que impede a execução de scripts injetados diretamente no HTML
- O uso de nonces garante que apenas scripts gerados pelo servidor (com o nonce correto) são executados - um atacante não consegue adivinhar o nonce
- A política default-src 'self' garante que recursos só podem ser carregados do próprio domínio, impedindo exfiltração de dados para servidores externos